Universidad Nacional de Loja

Facultad de la Energía, las Industrias
y los Recursos Naturales No Renovables

Carrera de Computación



Latencia de almacenamiento y velocidad de respuesta del software

Autor:

Isaac Alejandro Vire Poma

Ciclo/Paralelo

Tercer Ciclo Paralelo A

Materia

Estructura de Datos

Docente

Cristian R. Narváez Guillén

11 de octubre de 2026

Latencia de almacenamiento y velocidad de respuesta del software

Podemos definir la latencia de almacenamiento como el tiempo que transcurre entre solicitar un dato y recibirlo. Esto para el usuario se traduce en tiempo de espera y, por lo tanto, se puede percibir como la velocidad de respuesta de la aplicación. Cada lectura que debe viajar por la red, o esperar a un disco lento, suma a la demora total de una respuesta. Aquí es donde el uso adecuado y correcto del almacenamiento de una aplicación puede acortar este tiempo. La fuente principal [1] sostiene que hacer que la información persista en el navegador permite a los desarrolladores mejorar el rendimiento y ofrecer una mejor experiencia para el usuario. La estrategia es acercar el dato a quien lo usa, pues cuanto más cerca está, menos tiempo de espera se genera. Esta síntesis contrasta esa idea con una guía de almacenamiento de web.dev [2] y con las cifras de referencia de latencia [3].

1. Distintas latencias

No todas las ubicaciones de datos cuestan lo mismo. La lista clásica de latencias [3] nos muestra que cada nivel es órdenes de magnitud más lento que el anterior. Cabe resaltar que las cifras varían dependiendo del año y del hardware. Por ello esta tabla sirve solamente como una forma de visualizar el orden de magnitud.

Operación	Latencia	Equivalente en una aplicación web
Referencia a memoria principal	100 ns	Un objeto ya cargado en la página
Lectura aleatoria de 4 KB en SSD	150 µs	Acceso a datos guardados en el dispositivo
Ida y vuelta en un mismo centro de datos	500 µs	Petición a un servidor muy cercano
Paquete California–Países Bajos–California	150 ms	Petición a un servidor en otro continente

La diferencia entre el acceso local y el viaje intercontinental es de varios órdenes de magnitud. Por eso el uso de distintos medios de almacenamiento como el *local storage*, una funcionalidad del navegador que guarda pares clave-valor y los conserva tras recargar o cerrar la página [1], reduce la espera y evita el viaje de red, normalmente el componente más caro. Además, permite trabajar con conexión limitada o nula y ofrece hasta 5 MB por dominio, frente a los 4 KB de una cookie. Pero la latencia no depende solo de la distancia al dato, sino también de **cómo** se accede a él. Para entender esto, veamos brevemente algunas formas de almacenamiento.

2. Formas de almacenamiento

Hay distintas formas de almacenamiento que se pueden utilizar al trabajar en un entorno web, y cada una de ellas puede ser útil dependiendo de la situación en la que el desarrollador se encuentre.

sessionStorage es una forma de almacenamiento que se limita a la vida útil de la pestaña. Es síncrono, por lo que bloquea el subproceso principal; además, sirve para guardar información pequeña y específica, tiene 5 MB de almacenamiento y solo contiene cadenas.

localStorage almacena información mucho más allá de una pestaña, por lo que puede guardarla incluso si se cierra la pestaña o el navegador, y fuera de eso, no es diferente del Session Store.

Las cookies, por otro lado, a pesar de almacenar información, que es solo por medio de cadenas, tienen limitaciones, y deben almacenar poca información, pues de lo contrario cada solicitud web tendrá un tamaño mayor.

File System Access permite que los usuarios lean y editen archivos, por lo que está pensado para casos en donde se debe abrir un documento, leerlo y guardar los cambios realizados.

IndexedDB es para el almacenamiento de datos en general, es asíncrono y no bloquea el subproceso principal [2]. Por eso puede complementar a localStorage, este se reserva para datos pequeños que se necesitan de inmediato, e IndexedDB para volúmenes mayores.

Cuando el dato está en la red, cada petición suma decenas o cientos de milisegundos y la interfaz no puede mostrarlo hasta que llega. Si está en el dispositivo, la espera baja. Sin embargo, puede haber problemas con ciertas soluciones, por ejemplo, localStorage es síncrono, por lo que, mientras lee o escribe, el subproceso principal no atiende otras tareas, de modo que con datos grandes la interfaz puede congelarse [2].

3. Ejemplos

Ejemplo 1: preferencias del usuario sin consultar al servidor. Guardar el tema visual o el idioma localmente permite pintar la interfaz predeterminada de inmediato, sin esperar una respuesta remota.

```
localStorage.setItem('tema', 'oscuro');                // se guarda en el dispositivo
const tema = localStorage.getItem('tema') || 'claro';  // lectura local
document.body.className = tema;
// La interfaz se pinta de inmediato, sin peticion de red.
```

Ejemplo 2: caché de datos complejos con JSON. Como localStorage solo almacena texto, los objetos se convierten con `JSON.stringify()` y se recuperan con `JSON.parse()`.

```
async function obtenerUsuario() {
  const guardado = localStorage.getItem('usuario');
  if (guardado) {
    return JSON.parse(guardado);
  }
```

```
  const datos = await (await fetch('/api/usuario')).json();
  localStorage.setItem('usuario', JSON.stringify(datos)); // se guarda para la proxima vez
}
// Desde la segunda llamada se evita la peticion de red.
```

Ejemplo 3: elegir la persistencia según el caso (local o de sesión). Al guardar el nombre en `localStorage`, este lo mantiene tras cerrar y reabrir la página, mientras que la edad en `sessionStorage` se borra. Elegir bien evita volver a pedir datos que deberían sobrevivir, y evita acumular datos que no hace falta conservar:

```
localStorage.setItem('nombre', nombre);   // persiste entre sesiones
sessionStorage.setItem('edad', edad);     // se borra al cerrar la pestaña
    // Elegir bien evita peticiones repetidas y datos innecesarios.
```

4. Límites a considerar

Reducir la latencia no es gratuito. En el caso de localStorage, solo guarda cadenas (lo que puede dificultar el trabajo) y es susceptible a ataques como XSS, por lo que no debe guardar datos sensibles [1]. Además, los datos de un almacenamiento web son por defecto de tipo *best effort*, lo que significa que el navegador puede eliminarlos si el dispositivo tiene poco espacio, salvo que se solicite un almacenamiento persistente, por lo que incluso con formas de poder elegir los datos que se borran primero, por medio de buckets que ayudan en el estableciendo de un orden de eliminación, siempre hay que tener en cuenta la opción de un almacenamiento best effort o de un almacenamiento persistente a la hora de almacenar datos. Por otro lado, un dato local puede quedar desactualizado respecto al servidor, de modo que toda caché exige una política de caducidad o invalidación.

5. Conclusión

La velocidad de respuesta del software depende en gran parte de dónde viven los datos y de cómo se accede a ellos. Mover información usada con frecuencia de la red al almacenamiento local reduce la latencia percibida y habilita el uso sin conexión [1, 3]. Sin embargo, el acceso síncrono puede bloquear la interfaz y, con ella, el subproceso principal, por lo tanto, para volúmenes grandes las opciones asíncronas son preferibles [2]. Lo recomendable sería guardar cerca lo que se lee mucho y cambia poco, elegir el mecanismo según el tamaño y el uso, y consultar a la red lo que debe estar al día.

Con estas pautas es posible crear software que responda más rápido, siempre que el mecanismo de almacenamiento se elija según el tipo y el volumen de los datos. Existen muchas maneras de almacenar datos, y la mejor depende del software que se tenga en mente y del entorno de desarrollo. Como todas tienen límites, ya sea de tamaño, de bloqueo del subproceso principal o de persistencia, lo importante es saber qué almacenar y dónde, para que el usuario se beneficie de una respuesta rápida y confiable.

Referencias

[1] Semah, B. (trad. Areal, C.). «Cómo usar LocalStorage en JavaScript». *freeCodeCamp Español*, 7 may. 2024. https://www.freecodecamp.org/espanol/news/como-usar-localstorage-en-javascript/

[2] Google. «Storage for the web». *web.dev*. https://web.dev/articles/storage-for-the-web

[3] Bonér, J. (basado en trabajo de P. Norvig y J. Dean, ∼2012). «Latency Numbers Every Programmer Should Know». https://gist.github.com/jboner/2841832